

Minkowski Kart: A relativistic kart-racing game extending the OpenRelativity project

R. Christie^{a)}

Independent researcher

(Dated: 17 June 2026)

We present *Minkowski Kart*, an open-source arcade kart-racing game in which special-relativistic kinematics, optics, and dynamics are woven into ordinary gameplay. The game is built on the SuperTuxKart engine and borrows its relativistic visual model from the MIT Game Lab’s *OpenRelativity* toolkit, which popularised real-time relativistic rendering for the games *A Slower Speed of Light* and its successors. We describe three substantive extensions of that work. First, the apparent position of every object is computed from the *retarded* (emission-time) light-travel solution combined with relativistic aberration of the incoming ray, rather than from an instantaneous Lorentz contraction of static world geometry; this Terrell–Penrose construction is applied per object, so independently moving karts and projectiles each warp according to their own velocity. Second, because the apparent-position map is strongly nonlinear, we introduce *dynamic GPU mesh tessellation*: hardware tessellation-control and -evaluation shaders adaptively subdivide track and object geometry at render time, with the subdivision density driven both by proximity to the player and by a closed-form estimate of the local magnification produced by aberration. This removes the faceting and surface-clipping artefacts that otherwise force authors to ship pre-subdivided meshes. Third, the relativistic effects are made *playable*: a user-adjustable speed of light, a relativistic longitudinal mass that scales the engine force by $1/\gamma^3$, proper-time accumulation, and a suite of physics-themed power-ups (warp bubble, time dilation, anti-particle pair production, wormholes, gravitational-wave fronts) turn relativistic phenomena into game mechanics. We discuss the implementation, the pedagogical affordances of an interactive low- c world, and the limitations of the approach.

I. INTRODUCTION

Special relativity is notoriously difficult to build intuition for, precisely because its effects are imperceptible at everyday speeds. A productive teaching strategy is therefore to lower the speed of light c until relativistic effects dominate the visible world, and to let students move through that world interactively.^{3,4} The most influential modern realization of this idea is the MIT Game Lab’s *OpenRelativity* toolkit¹ and the game *A Slower Speed of Light*, in which a first-person player walks through a static environment whose apparent geometry and color are transformed in real time as the in-game c is gradually reduced toward the player’s running speed. *OpenRelativity* is distributed as a Unity plugin² and has been widely adopted for classroom demonstrations and outreach.

OpenRelativity makes two simplifying assumptions that are well suited to its walking-simulator setting but limiting for a fast, multi-object arcade game. (i) The relativistic vertex transform is applied to a *static* world in the player’s instantaneous rest frame: the player is treated as moving through fixed scenery, and there is no general mechanism for many independent objects to each carry their own velocity into the optical transform. (ii) The transform is evaluated only at existing mesh vertices, so faithful rendering requires geometry that is already finely subdivided; coarse meshes visibly facet because the nonlinear warp is interpolated linearly across

large triangles.

We should be clear about what is and is not new here. The underlying optical model—placing each object at its retarded (light-emission) position and aberrating the incoming ray into the observer’s frame—is the standard “photographic” approach to relativistic visualization, developed over several decades and used, in one form or another, by essentially all of the prior art.^{1,3,4,8,9} The Terrell–Penrose insight that this produces apparent *rotation* rather than visible contraction is older still.^{5–7} Our contribution is not a new optical model but an engine architecture that makes that model work in a fast, many-body, competitively played game on externally-authored content. Concretely, the novel elements are (a) applying the retarded-time/aberration transform *per object*, each with its own velocity, so independently moving karts and projectiles warp correctly relative to a moving observer; (b) a dynamic GPU tessellation scheme whose subdivision density is driven by a closed-form estimate of the local aberration magnification, which lets the nonlinear warp render cleanly on arbitrary track geometry; and (c) a playable relativistic *dynamics* layer (adjustable c , $1/\gamma^3$ longitudinal response, proper-time accounting, velocity clamping) that turns the effects into game mechanics rather than a passive visual overlay.

Minkowski Kart is an open-source game that keeps *OpenRelativity*’s physical model and even reuses its spectral Doppler-shift code, but rebuilds the relativistic pipeline around the demands of a racing game: dozens of objects moving at appreciable fractions of c relative to one another, on artist-authored tracks of arbitrary tessellation, rendered through two graphics back-

^{a)} Electronic mail: slipperyfish223@gmail.com



FIG. 1. An all-AI race on the Möbius Trip track. The heads-up display reports the live relativistic telemetry of the lead kart—coordinate speed v , the ratio $\beta = v/c$, accumulated proper time τ , and the in-game speed of light c . Even at $\beta \approx 0.57$ the track surface and trailing opponents are already visibly warped by aberration and retarded-time displacement (Sec. IV).

ends (OpenGL and Vulkan). The game is a fork of SuperTuxKart¹⁰, an established open-source kart racer, and is released under the GPLv3.

This paper documents the physics model and the three principal extensions over OpenRelativity: a per-object retarded-time/aberration optical model (Sec. IV), dynamic GPU mesh tessellation driven by the local non-linearity of the warp (Sec. V), and a set of relativistic *dynamics* that make the effects controllable as game-play (Sec. VI). We also describe the relativistic power-up system (Sec. II) and the Doppler/searchlight color shift (Sec. VII), and we close with pedagogical observations and limitations.

II. THE GAME

Minkowski Kart plays as a conventional arcade kart racer: the player drives a kart around a closed circuit, collects power-up boxes, and competes against human or AI opponents. What differs is that the in-game speed of light is small enough to be reached during normal driving. The default coordinate speed of light is $c = 35$ u/s (game units per second; roughly the top speed of a boosted kart), and the player may set it anywhere in the range 15–1000 u/s from the “Relativity” options menu, with a hard cap on coordinate speed at $\beta_{\max} = 0.98$ by default. As a kart approaches c , opponents and scenery visibly distort, their colors Doppler-shift, and the player’s own clock runs slow relative to coordinate time.

Each power-up is themed on a relativistic or particle-physics concept (Table I), so that the act of using an item is also an encounter with the idea behind it. The game runs on desktop (Windows, Linux, macOS) and supports online multiplayer in which the relativistic rules— c , $\beta_{\max}$, and the power-up boost multiplier—are set by the server and synchronized to all clients.

TABLE I. Representative relativistic power-ups. The full item logic lives in `src/items/` and item weights in `data/powerup.xml`.

Power-up	Physical motif
Warp Bubble	Local boost to c ($10\times$ normal) and shield
Black Hole	Homing mass with lensing and orbital capture
Wormhole	Linked pair of traversable spacetime portals
Time Dilation	Expanding wavefront that lowers others’ c
Anti-Karticle	Mirror clone; annihilates in a pair-production flash
Super Position	Global spacetime “collapse” and world pulse
Photon	High-speed tether / Doppler blast
Maxwell–Boltzmann	Gaussian velocity kicks aimed at the leader

III. RELATIVISTIC KINEMATICS WITH AN ADJUSTABLE c

All relativistic bookkeeping is centralized in a single module (`src/relativity/relativity_math.cpp`). For a kart with signed speed v along its motion and an effective speed of light c , the standard invariants

$$\beta = \frac{|v|}{c}, \quad \gamma = \frac{1}{\sqrt{1 - \beta^2}} \quad (1)$$

are clamped to $\beta < 1$ for numerical safety. Proper time is accumulated each physics tick from the coordinate time step Δt as

$$\Delta\tau = \frac{\Delta t}{\gamma}, \quad (2)$$

so each kart carries both a coordinate clock and a proper clock; the running difference is the twin-paradox bookkeeping made literal. The speed of light is not a fixed constant but a smoothly ramped state variable: when a power-up changes a kart’s local c (for example the warp bubble, which sets it to $10\times$ the baseline), the value eases over ~ 1 s with a smoothstep so that the optical distortion does not snap discontinuously.

IV. OPTICS: RETARDED POSITION AND ABERRATION

The central visual question is: where does an object *appear* to a moving observer? Minkowski Kart answers it the same way a camera does, in two steps, and—unlike OpenRelativity’s contraction-of-static-geometry approach—it applies the construction to each object using that object’s own velocity.

a. Retarded (emission-time) position. Light reaching the observer now was emitted in the past, when the object was elsewhere. For an object at relative position $\mathbf{r}$

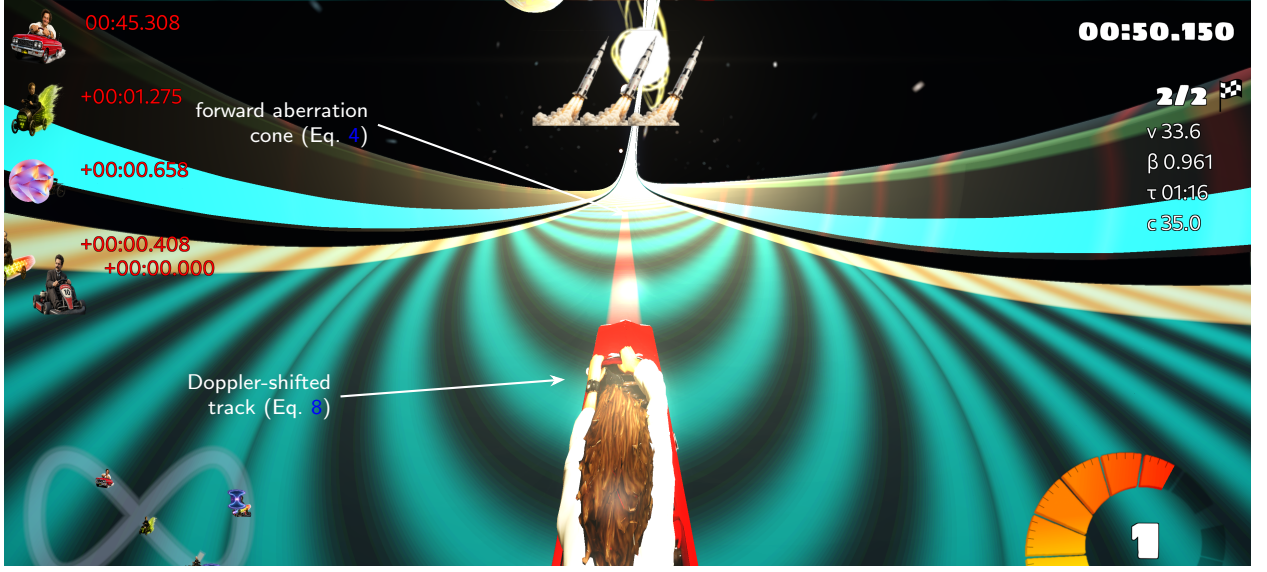


FIG. 2. Strong relativistic distortion at $\beta = 0.961$ ($\gamma \approx 3.7$) on the Möbius Trip track. Forward aberration squeezes the entire forward hemisphere into a narrow cone ahead of the kart (the “headlight” effect of Eq. (4)), bowing the straight track into concentric arcs, while the relativistic Doppler/searchlight shift of Eq. (8) paints the surface in shifted bands (Sec. VII). The geometry is resolved without faceting because the track is dynamically tessellated on the GPU (Sec. V). The HUD reads $v = 33.6$, $\beta = 0.961$, $c = 35$.

moving with velocity $\mathbf{u}$, the emission time offset $\Delta t_e \leq 0$ satisfies $|\mathbf{r} + \mathbf{u} \Delta t_e| = -c \Delta t_e$, i.e. the quadratic

$$(\mathbf{u} \cdot \mathbf{u} - c^2) \Delta t_e^2 + 2(\mathbf{r} \cdot \mathbf{u}) \Delta t_e + \mathbf{r} \cdot \mathbf{r} = 0. \quad (3)$$

We take the most-recent past root and displace the object to its emission-time position $\mathbf{r}' = \mathbf{r} + \mathbf{u} \Delta t_e$. This is the geometric origin of the Terrell–Penrose effect: because near and far parts of a fast object emitted the light reaching the eye at different times, a passing object appears *rotated* rather than merely contracted.^{5,6}

b. Aberration of the incoming ray. The direction $\hat{\mathbf{d}}$ of the emission-time ray is then transformed from the world frame into the observer’s frame by the relativistic aberration map, which for an observer with velocity βc and Lorentz factor γ beams forward rays toward the direction of motion (the “headlight” effect):

$$\hat{\mathbf{d}}_{\text{obs}} = \frac{\frac{1}{\gamma} \hat{\mathbf{d}} + \left[\frac{\gamma}{\gamma + 1} (\beta \cdot \hat{\mathbf{d}}) + 1 \right] \beta}{1 + \beta \cdot \hat{\mathbf{d}}}. \quad (4)$$

The apparent position is finally $\mathbf{p}_{\text{obs}} = \mathbf{p}_{\text{eye}} + |\mathbf{r}'| \hat{\mathbf{d}}_{\text{obs}}$.

A deliberate design choice—documented in the shader source—is that *no explicit Lorentz contraction* is applied to positions, normals, or tangents. What a camera records is fully captured by the retarded position plus aberration; coordinate length contraction is a statement about simultaneity in a chosen frame, and adding it on top of the optical transform would double-count the effect.⁷ Lighting is therefore evaluated with unmodified

world-space normals, because light transport happens in the world frame.

Equations (3)–(4) are implemented identically on the CPU (for gameplay queries such as “where does the track surface appear?” used by the anti-clipping logic) and on the GPU (for rendering), so the two never disagree. The GPU version lives in `data/shaders/utils/relativity_visual.vert` and is included by every relativistic vertex/tessellation stage:

```
// per-vertex apparent position (GLSL, abridged)
relative = world_position - u_observer_pos;
relative = retardedEmissionPosition(relative,
    object_velocity);
vec3 d_obs = worldDirToObserverDir(normalize(relative));
return u_observer_pos + length(relative) * d_obs;
```

V. DYNAMIC MESH TESSELLATION

The apparent-position map of Sec. IV is strongly non-linear in world position. If it is evaluated only at the corners of a coarse triangle and then interpolated linearly across the face—as it is when a static vertex shader warps an artist’s mesh—straight edges that should bow become visibly faceted, and the warped track surface develops cracks and clips through karts. In OpenRelativity this is handled by authoring heavily subdivided geometry. That is impractical for a racing game whose tracks are large, externally authored, and shared with the non-relativistic engine.

Minkowski Kart instead subdivides geometry *dynamically on the GPU* at render time, using the hard-

ware tessellation stages (`data/shaders/sp_tess.tesc` and `sp_tess.tese`). The tessellation-control shader assigns each triangle edge a subdivision level, and the tessellation-evaluation shader re-applies the relativistic apparent-position map (3)–(4) at every newly generated vertex, so the warp is sampled where it is actually non-linear instead of being interpolated across it.

a. Distance-adaptive base level. The base subdivision keeps the rendered edge length below a target. Rather than cutting tessellation off at a fixed radius (which leaves large, rigidly warped triangles in the distance), the target edge length *grows* with distance from the player, so geometry keeps subdividing everywhere, just more coarsely far away:

$$\ell_{\text{target}}(d) = \ell_0 \max\left(1, \frac{d}{R_0}\right) / A(\beta), \quad (5)$$

with near-field target $\ell_0 = 0.1$, falloff scale $R_0 = 10$ world units, and the tessellation level for an edge of length ℓ taken as $\text{clamp}(\ell/\ell_{\text{target}}, 1, 64)$.

b. Aberration-aware amplification. The factor $A(\beta) \geq 1$ in Eq. (5) is the novel ingredient. Aberration magnifies image space anisotropically: rays *behind* a forward-moving observer are spread out by a factor $\sim 1/(1 + \beta \cdot \hat{\mathbf{d}})$, so geometry there must be subdivided more finely to keep the warp smooth. We estimate the local magnification directly from the edge midpoint direction $\hat{\mathbf{d}}$:

$$A(\beta) = \text{clamp}\left(\frac{1}{\max(1 + \beta \cdot \hat{\mathbf{d}}, \frac{1}{8})}, 1, 8\right). \quad (6)$$

This raises the subdivision density precisely where the optical transform is most expansive (behind the kart at high β) and relaxes it elsewhere, spending the tessellation budget where the eye can see the difference.

c. Crack-free guarantee. Both Eq. (5) and Eq. (6) depend *only on the two endpoints of an edge* (and on global observer state), never on the interior or the opposite vertex of a triangle. Two triangles that share an edge therefore compute exactly the same level for it, and the tessellator produces matching vertices on both sides—so no T-junction cracks appear along shared edges, a property that is otherwise easy to violate with adaptive tessellation.

d. Where it matters. The benefit of dynamic tessellation scales with the size of the base triangles relative to the curvature of the warp: it is dramatic on coarse or distant geometry—a large ground or water plane subdivided into a handful of huge triangles will visibly facet and clip without it—and negligible on geometry that is already finely authored. On a tightly subdivided track such as the Möbius Trip circuit used for our figures, disabling tessellation leaves the warped surface nearly indistinguishable even under magnification, because the artist’s mesh is fine enough to sample the warp adequately on its own. The point of the adaptive scheme is precisely that the

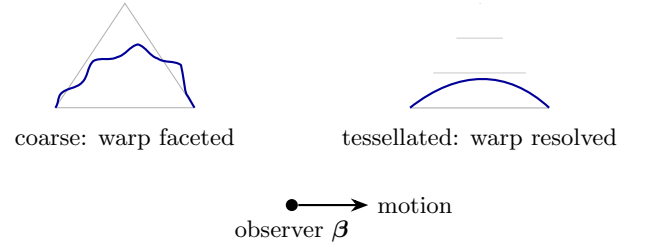


FIG. 3. Schematic of dynamic tessellation. The nonlinear apparent-position warp (blue) is poorly captured across a coarse triangle (left) but well resolved once the triangle is adaptively subdivided on the GPU and the warp is re-evaluated at the new vertices (right). Subdivision density rises with proximity to the observer and with the aberration magnification $A(\beta)$ of Eq. (6).

engine need not assume either case: it spends subdivision only where the local triangle size and the aberration magnification of Eq. (6) demand it, so the same renderer handles a coarse hand-made plane and a dense imported mesh without per-asset tuning.

e. Other anti-clipping modes. Tessellation is one of several user-selectable strategies for reconciling the warped visual surface with the unwarped collision mesh. The Relativity menu also exposes a cheap height-correction mode (adjusting visual height against the physical track), a tangent-velocity-projection mode, and a combined “tessellation + height correction” mode; the enhanced tessellation path requires desktop GLSL 4.00+ or GLSL ES 3.20+.

VI. RELATIVISTIC DYNAMICS AS GAMEPLAY

Where OpenRelativity is primarily an optical layer over an otherwise Newtonian world, Minkowski Kart also makes the *dynamics* relativistic, because in a racing game the player must *feel* the speed limit, not merely see it. Two mechanisms accomplish this. The first is relativistic inertia: a force applied along the direction of motion accelerates a body as if its mass were enhanced by the longitudinal factor γ^3 , so the engine force that would increase a kart’s forward speed is scaled down,

$$F_{\parallel} \rightarrow \frac{F_{\parallel}}{\gamma^3}. \quad (7)$$

Because $1/\gamma^3$ collapses toward zero as $\beta \rightarrow 1$ (Fig. 5), acceleration fades smoothly into a wall at c with no artificial cruise-speed equilibrium, while braking and reverse forces, which reduce speed, are left unscaled so the kart always remains controllable. The second mechanism is a final hard guard: the coordinate velocity is clamped to $|\mathbf{v}| \leq \beta_{\text{max}}c$ with its direction preserved, which keeps the simulation numerically stable and enforces the chosen ceiling even through collisions and power-up boosts that might otherwise overshoot it.

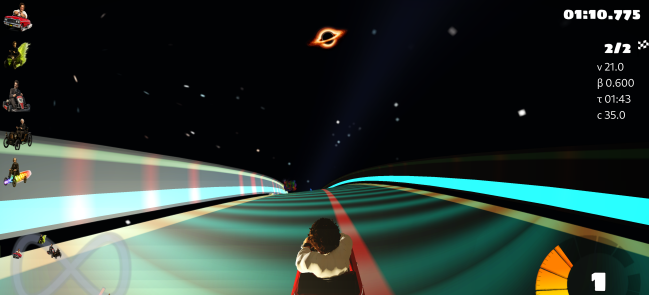


FIG. 4. A gravitational-lensing item (the glowing accretion ring overhead) seen mid-race at $\beta = 0.600$. The black-hole and wormhole power-ups render as lensing and accretion VFX backed by shared physics constants (Sec. II); they are evocative effects rather than solutions of the field equations (Sec. X).

a. Time dilation as a weapon. The Time Dilation power-up emits an expanding wavefront (radius $R = 75$ u reached in 3 s, i.e. a front speed of 25 u/s) that lowers the effective c of every kart it passes, exactly when the visible ripple reaches them; gameplay timing and the screen-space ripple are driven from the same constants so cause and effect stay locked together. Several other items (black hole, wormhole, anti-particle pair production) are similarly backed by shared physics constants so that the visual and the mechanical effect remain consistent.

VII. DOPPLER AND SEARCHLIGHT COLOR SHIFT

Color shift is the one place where Minkowski Kart reuses OpenRelativity’s code essentially verbatim, with attribution: the spectral RGB $\rightarrow$ XYZ decomposition into Gaussian primaries, the wavelength rescaling, and the inverse transform back to displayable RGB are taken from OpenRelativity’s color shader.^{1,2} The relativistic Doppler/searchlight factor for a ray of view direction $\hat{\mathbf{d}}$ seen by an observer with reduced velocity β is

$$s = \gamma(1 - \beta \cdot \hat{\mathbf{d}}), \quad (8)$$

which blueshifts ($s < 1$) objects ahead and redshifts those behind, with intensity rescaled by s^{-3} to respect the relativistic beaming of spectral radiance. Minkowski Kart adds a circular “scanner” window at screen center that is left unshifted (and rendered as a monochrome instrument readout) so the player retains a usable view forward while the periphery shifts—an ergonomic compromise the walking-simulator setting did not require.

VIII. ARCHITECTURE AND PORTABILITY

The relativistic state for the local observer— β , γ , c , and observer position—is gathered once per frame into

TABLE II. Selected analytic checks from the start-up regression suite (`relativity_math.cpp`), here with $c = 80$. Each is asserted to hold to $\sim 10^{-5}$ in every build.

Quantity	Input	Required value
γ	$\beta = 0.6$	1.25
$\Delta\tau = \Delta t/\gamma$	$\Delta t = 1, \gamma = 2$	0.5
$F_{\parallel}/\gamma^3$	$F = 100, \beta = 0.6$	51.2
velocity clamp $\beta_{\max}c$	$ \mathbf{v} = 100, \beta_{\max}c = 78.4$	78.4
$ \hat{\mathbf{d}}_{\text{obs}} $	any ray	1 (norm preserved)

an “observer snapshot” and uploaded as shader uniforms shared by the OpenGL “SP” renderer and the Vulkan “GE” renderer, so the same physics drives both backends. Because the optical transform and the gameplay queries call the identical math, on-screen appearance and the anti-clipping ray casts agree by construction. In networked play a server can broadcast relativity rules (baseline c , $\beta_{\max}$, power-up multiplier) that override the local configuration, keeping all clients in a common relativistic regime.

IX. VALIDATION

Because the same routines drive both the visible distortion and the gameplay physics, an error in the relativity module would be simultaneously a physics bug and a rendering bug. We guard against regressions with a suite of assertions that run at start-up in every build (`Relativity::unitTesting` in `relativity_math.cpp`); each checks the implementation against a hand-computed analytic value. Representative cases are collected in Table II: the Lorentz factor at $\beta = 0.6$ is required to equal 1.25, the proper-time element $\Delta\tau = \Delta t/\gamma$ to halve at $\gamma = 2$, the longitudinal force factor $1/\gamma^3$ to reduce a unit force to 0.512 at $\beta = 0.6$, and the velocity clamp to return exactly $\beta_{\max}c$ when fed a superluminal input. The aberration map is checked to be distance-preserving (it only redirects rays), and to beam a forward off-axis point toward the direction of motion, consistent with Eq. (4).

The model the game runs is summarized in Fig. 5, which plots the four speed-dependent quantities a player actually feels and sees: the time-dilation factor γ , the proper-clock rate $1/\gamma$, the longitudinal force response $1/\gamma^3$, and the head-on Doppler/searchlight factor $s = \gamma(1 + \beta)$ of Eq. (8). The sharp upturn of γ and the equally sharp collapse of $1/\gamma^3$ near $\beta = 1$ are the quantitative origin of the “wall at c ” the player experiences: past $\beta \approx 0.9$, almost no additional engine force converts into coordinate speed.

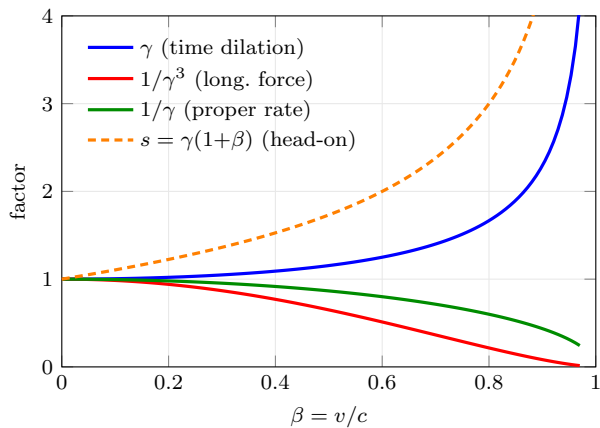


FIG. 5. The speed-dependent quantities implemented by the relativity module and verified by the checks in Table II. The divergence of γ and the collapse of the longitudinal force factor $1/\gamma^3$ as $\beta \rightarrow 1$ are what make the speed of light feel like a hard ceiling during play, while the head-on Doppler factor s drives the color shift of Fig. 2.

X. PEDAGOGICAL USE AND LIMITATIONS

By placing the speed of light within reach of an everyday action—driving—the game makes several relativistic phenomena directly manipulable: aberration and Terrell rotation (watch an opponent appear to swing around as it overtakes), relativistic Doppler shift, the γ^3 resistance to further acceleration, and proper-time lag between coordinate and on-board clocks. Unlike a fixed demonstration, the player sets c and explores the continuum between the Newtonian and ultrarelativistic limits.

The model has clear limitations, appropriate to flag for instructors. The dynamics are special-relativistic only; the “black hole,” “wormhole,” and “gravitational-wave” items are evocative visual effects, not solutions of the field equations. The optical model captures retarded position and aberration but not relativistic shadowing or multi-bounce light transport. And the adjustable c is a single global (or per-kart) scalar, so the game does not model an observer-dependent simultaneity beyond what the optical transform encodes. These are reasonable simplifications for an arcade game, and each is an opening

for classroom discussion of what the simulation does and does not assert.

XI. CONCLUSION

Minkowski Kart shows that the relativistic-rendering ideas pioneered by OpenRelativity can be carried from a single-observer walking simulator into a fast, many-object, competitively-played game. The three extensions that make this possible—a per-object retarded-time-plus-aberration optical model, dynamic GPU tessellation that resolves the nonlinear warp on arbitrary geometry, and a playable relativistic dynamics with adjustable c —may be useful beyond this particular game wherever real-time relativistic visuals must run on externally-authored content. The source is available under the GPLv3.

ACKNOWLEDGMENTS

We thank the SuperTuxKart contributors for the underlying engine and the MIT Game Lab for OpenRelativity, whose physical model and color-shift code this work builds upon.

- ¹Z. W. Sherin, R. Cheu, P. Tan, and G. Kortemeyer, “Visualizing relativity: The OpenRelativity project,” *Am. J. Phys.* **84**, 369–374 (2016).
- ²MIT Game Lab, “OpenRelativity,” <https://github.com/MITGameLab/OpenRelativity> (accessed 2026).
- ³C. M. Savage, A. Searle, and L. McCalman, “Real Time Relativity: Exploratory learning of special relativity,” *Am. J. Phys.* **75**, 791–798 (2007).
- ⁴U. Kraus, “First-person visualizations of the special and general theory of relativity,” *Eur. J. Phys.* **29**, 1–13 (2008).
- ⁵J. Terrell, “Invisibility of the Lorentz contraction,” *Phys. Rev.* **116**, 1041–1045 (1959).
- ⁶R. Penrose, “The apparent shape of a relativistically moving sphere,” *Proc. Cambridge Philos. Soc.* **55**, 137–139 (1959).
- ⁷V. F. Weisskopf, “The visual appearance of rapidly moving objects,” *Phys. Today* **13**(9), 24–27 (1960).
- ⁸P.-K. Hsiung and R. H. P. Dunn, “Visualizing relativistic effects in spacetime,” in *Proceedings of the 1989 ACM/IEEE Conference on Supercomputing* (ACM, New York, 1989), pp. 597–606.
- ⁹D. Weiskopf, U. Kraus, and H. Ruder, “Searchlight and Doppler effects in the visualization of special relativity: A corrected derivation of the transformation of radiance,” *ACM Trans. Graph.* **18**, 278–292 (1999).
- ¹⁰SuperTuxKart contributors, “SuperTuxKart,” <https://github.com/supertuxkart/stk-code> (accessed 2026).